

Ingeniería del Software II

Guía 4.

Ejercicio 2. Analizar y justificar la validez de lo siguiente:

- $\Box\Box\phi \rightarrow \Box\phi$
- $\Box(\phi \wedge \psi) \rightarrow \Box\phi \wedge \Box\psi$
- $\Diamond(\phi \vee \psi) \rightarrow \Diamond\phi \vee \Diamond\psi$

$$\Box\Box\phi \rightarrow \Box\phi$$

$$\Leftrightarrow \neg(\Box\Box\phi) \vee \Box\phi \quad \Box\Box\phi \equiv \Box\phi$$

$$\Leftrightarrow \neg(\Box\phi) \vee \Box\phi$$

$$\Leftrightarrow \neg(\forall j \geq 0 \ \sigma[0:j] \models \phi) \vee \forall j \geq 0 \ \sigma[0:j] \models \phi$$

$$\Leftrightarrow \text{True.}$$

$$\Box(\phi \wedge \psi) \rightarrow \Box\phi \wedge \Box\psi$$

$$\Leftrightarrow \neg(\Box(\phi \wedge \psi)) \vee (\Box\phi \wedge \Box\psi) \quad \Box(\phi \wedge \psi) \equiv \Box\phi \wedge \Box\psi$$

$$\Leftrightarrow \neg(\Box\phi \wedge \Box\psi) \vee (\Box\phi \wedge \Box\psi)$$

$$\Leftrightarrow \text{True}$$

$$\Diamond(\phi \vee \psi) \rightarrow \Diamond\phi \vee \Diamond\psi$$

$$\Leftrightarrow \neg(\Diamond(\phi \vee \psi)) \vee (\Diamond\phi \vee \Diamond\psi) \quad \Diamond(\phi \vee \psi) \equiv \Diamond\phi \vee \Diamond\psi$$

$$\Leftrightarrow \neg(\Diamond\phi \vee \Diamond\psi) \vee (\Diamond\phi \vee \Diamond\psi)$$

$$\Leftrightarrow \text{True}$$

Ejercicio 3. Demuestre o refute la validez de la fórmula LTL $\Box(p \wedge \Diamond q) \leftrightarrow \Box(p \vee q)$. Justifique formalmente su respuesta.

Falso. Vale la ida pero no la vuelta. Prueba:

$$\Box(p \wedge \Diamond q) \leftarrow \Box(p \vee q)$$

Veamos la ida y luego la vuelta

$$\Box(p \wedge \Diamond q) \rightarrow \Box(p \vee q)$$

$$\forall i \geq 0 \ \sigma[i..] \models (p \wedge \Diamond q) \rightarrow \forall i \geq 0 \ \sigma[i..] \models (p \vee q)$$

$$\forall i \geq 0 \ \sigma[i..] \models p \wedge \sigma[i..] \models \Diamond q \rightarrow \forall i \geq 0 \ \sigma[i..] (\exists j \geq 0 \ \sigma[j..] \models q \wedge \forall k \ 0 \leq k < j \ \sigma[k..] \models p)$$

$$\forall i \geq 0 \ \sigma[i..] \models p \wedge \sigma[i..] \models (\exists j \geq 0 \ \sigma[j..] \models q)$$

$$\rightarrow \forall i \geq 0 \ \sigma[i..] (\exists j \geq i \ \sigma[j..] \models q \wedge \forall k \ i \leq k < j \ \sigma[k..] \models p)$$

$$\forall i \geq 0 (\sigma[i..] \models p \wedge \exists j \geq i \ \sigma[j..] \models q)$$

$$\rightarrow \forall i \geq 0 (\exists j \geq i (\sigma[j..] \models q \wedge \forall k \ i \leq k < j \ \sigma[k..] \models p))$$

Claramente si se cumple el precedente $\forall i \geq 0 (\exists j \geq i \ \sigma[j..] \models q \wedge \sigma[i..] \models p)$ no dice que se cumple que $\forall i \geq 0 \ \sigma[i..] \models p$ dice que se cumple que $\forall k \ i \leq k \ \sigma[k..] \models p$ entonces vale que $\forall i \geq 0 \exists j \geq i \forall k \ i \leq k < j \ \sigma[k..] \models p$ y $\forall i \geq 0 \exists j \geq i \ \sigma[j..] \models q$ es la primera condición del consecuente, por ende si vale el precedente vale el consecuente y se cumple el implica.

Claramente la vuelta no vale, ya que $\forall i \geq 0 \exists j \geq i \forall k \ i \leq k < j \ \sigma[k..] \models p$ solo garantiza que vale p para cierto rango, de hecho si $k=j$ es trivialmente válido y no implica que $\forall i \geq 0 \ \sigma[i..] \models p$.

Ejercicio 4. Demuestre formalmente que *strong fairness* implica *weak fairness*.

Queremos ver que:

$$(\Box \Diamond \phi \rightarrow \Box \Diamond \phi) \rightarrow (\Diamond \Box \phi \rightarrow \Box \Diamond \phi)$$

Sólo nos interesa ver que si vale el precedente vale el consecuente. Si vale el precedente entonces o $\neg \Box \Diamond \phi$ o $\Box \Diamond \phi$, en el segundo caso esto también haría verdadero al consecuente. Así que veamos el caso en que $\neg \Box \Diamond \phi$.

$$\neg \Box \Diamond \phi$$

$$\Rightarrow \neg (\forall i \geq 0 \exists j \geq i \ \sigma[j..] \models \phi)$$

$$\Rightarrow \exists i \geq 0 \forall j \geq i \sigma[j..] \models \phi$$

$$\Diamond \Box \neg \phi$$

y claramente si se da $\Diamond \Box \neg \phi$ entonces no se puede dar $\Box \Box \phi$ por ende $\neg \Diamond \Box \phi$ y se cumple el consecuente.

Ejercicio 5. Intente expresar en lógica temporal lineal las siguientes propiedades:

- "Si ϕ es cierto durante la ejecución del programa, eventualmente ocurrirá ψ "
- "No es posible que ϕ y ψ ocurran simultáneamente durante la ejecución del programa"
- "Cada vez que ϕ sea cierto, ψ también es"
- " ϕ no deja de ocurrir, al menos hasta que ψ ocurra"

• $\Box \phi \rightarrow \Diamond \psi$ o $\Box (\phi \rightarrow \Diamond \psi)$ (depende de la interpretación)

• $\Box \neg (\phi \wedge \psi) \equiv \neg \Diamond (\phi \wedge \psi)$

• $\Box (\phi \rightarrow \psi)$

• $\phi W \psi$ (la diferencia con el $\phi U \psi$ es que obliga a que ψ ocurra mientras que $\phi W \psi = (\phi U \psi) \vee \Box \phi$)

Ejercicio 6. El operador "while" se puede definir como

$$\sigma \models \phi W \psi \text{ sii } \forall i \geq 0, \text{ si } \forall j, 0 \leq j \leq i, \sigma[j..] \models \psi \text{ entonces } \sigma[i..] \models \phi.$$

1. ¿Puede expresar el operador W en términos de U y operaciones booleanas?
2. ¿Puede expresar el operador U en términos de W y operaciones booleanas?

1. $\Box \phi \vee (\phi U \neg \psi)$

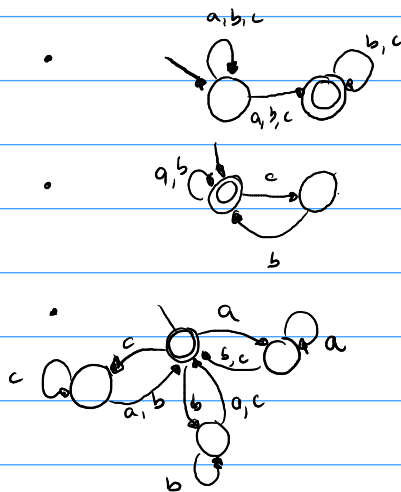
2. $(\phi W \neg \psi) \wedge \Diamond \psi$

Ejercicio 7. Dadas dos fórmulas LTL ϕ y ψ , sea $\phi Z \psi$ el lenguaje conteniendo todas las secuencias $\rho \in (2^{PA})^\omega$ tales que, para dos posiciones distintas en las cuales ϕ es válida, ψ es válida en una tercera posición entre medio de las otras dos. Dé una definición de $\phi Z \psi$ usando los operadores de lógica LTL.

$$\phi Z \psi \equiv \Box ((\phi \wedge \Diamond \phi) \rightarrow \Diamond (\neg \phi \vee (\psi \wedge \neg \phi)))$$

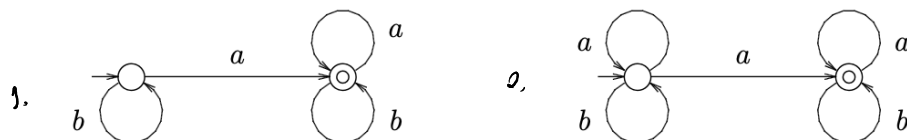
Ejercicio 8. Defina autómatas de Büchi para los siguientes lenguajes sobre el alfabeto $\{a, b, c\}$:

- conjunto de cadenas infinitas con una cantidad finita de a ,
- conjunto de cadenas infinitas en que cada ocurrencia de c viene inmediatamente seguida de una ocurrencia de b ,
- conjunto de cadenas infinitas que no terminen en una secuencias infinitas de a ni en una de b ni en una de c ,
- conjunto de cadenas infinitas con cantidades finitas de a , b y c .



• No se puede

Ejercicio 9. Demostrar que los siguientes autómatas aceptan el mismo lenguaje ω -regular.



Dar la expresión ω -regular del ω -lenguaje definido por ambos autómatas.

$$L(A_1) = b^* a (a+b)^\omega$$

$$L(A_2) = (a+b)^* a (a+b)^\omega$$

$$\text{Claramente } b^* a (a+b)^\omega \subseteq (a+b)^* a (a+b)^\omega$$

Ahora veamos la vuelta, ¿? $(a+b)^* a (a+b)^\omega \subseteq b^* a (a+b)^\omega$ entonces

$$(a+b)^* a (a+b)^\omega \cap \overline{b^* a (a+b)^\omega} = \emptyset$$

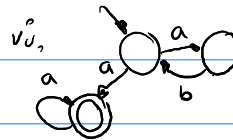
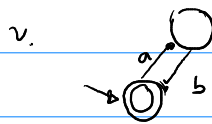
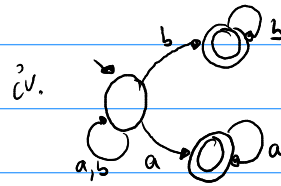
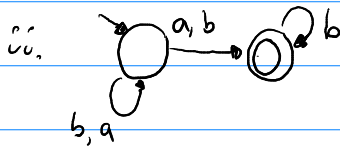
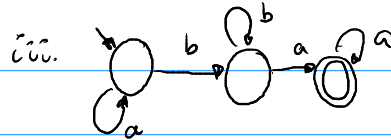
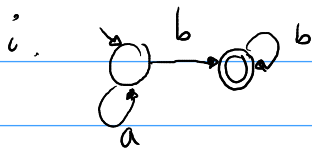
$$(a+b)^* a (a+b)^\omega \cap b^\omega = \emptyset$$

$$\text{Entonces } (a+b)^* a (a+b)^\omega = b^* a (a+b)^\omega$$

Ejercicio 10. Para todas las expresiones ω -regulares dadas o calculadas en los Ejercicios 8, 9 y 10 del Trabajo práctico 3, defina los autómatas de Büchi que aceptan el mismo lenguaje.

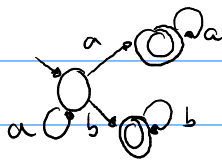
Ejercicio 8. Considere el alfabeto $\Sigma = \{a, b\}$. Determine si las siguientes propiedades son de *safety*, *liveness*, ambas, o ninguna. Justifique la respuesta:

- i. a^*b^ω iii. $a^*b^+a^\omega$ v. $(ab)^\omega$
 ii. $(b+a)^+b^\omega$ iv. $(a+b)^*(a^\omega + b^\omega)$ vi. $(ab)^*a^\omega$

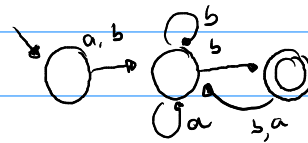


Ejercicio 9. Para cada propiedad P del Ejercicio 8 que no sea de *safety* extiéndala agregando el menor conjunto de trazas de manera que lo sea. Use expresiones ω -regulares. Justifique su respuesta.

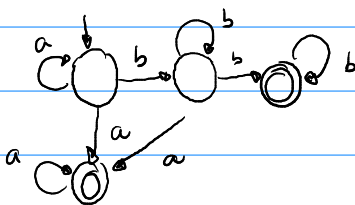
i. $a^*(a^\omega + b^\omega)$



ii. $(b+a)((b+a)^*b)^\omega$



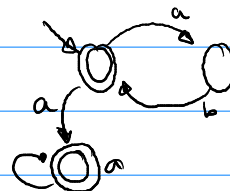
iii. $a^*(b^+a^\omega + b^\omega + a^\omega)$



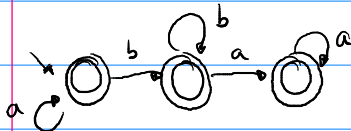
iv. $(a+b)^\omega$



v. $(ab)^*(a^\omega + (ab)^\omega)$



alt.

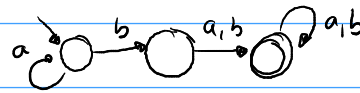
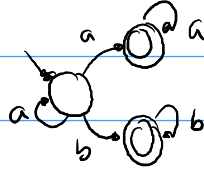
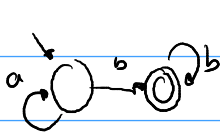


Ejercicio 10. Para cada propiedad P del Ejercicio 8 que no sea ni de safety ni de liveness, dé una propiedad de safety S y una de liveness L tal que $S \cap L = P$. Justifique su respuesta.

i. $P = a^* b^w$

$S = a^*(a^w + b^w)$

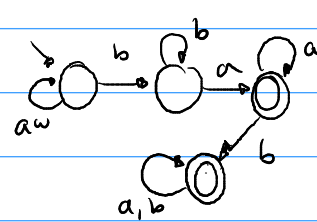
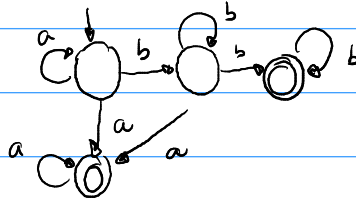
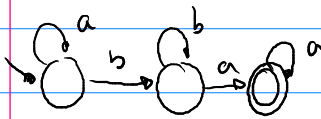
$L = (ba + b + a^*b)(a+b)^w$



iii. $P = a^* b^+ a^w$

$S = a^*(b^+ a^w + b^w + a^w)$

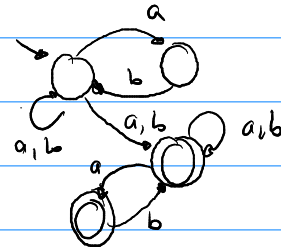
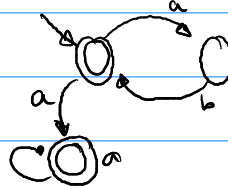
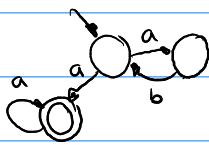
$L = a^* b^+ a^w \cup ba b (a+b)^w$



vi. $P = (ab)^* a^w$

$S = (ab)^*(a^w + (ab)^w)$

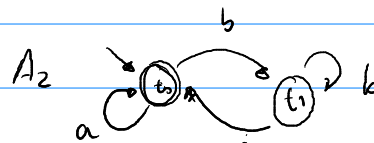
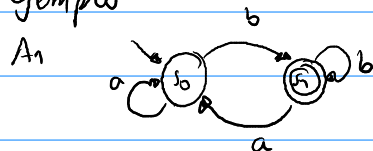
$L = (a + b + ab)^*(ba + b + a)^w$



Ejercicio 11. Averigüe cómo se construye, dados dos autómatas de Büchi A_1 y A_2 , el autómata $A_{A_1 \cap A_2}$.

Utilizando el producto cartesiano y un estado de estado final.

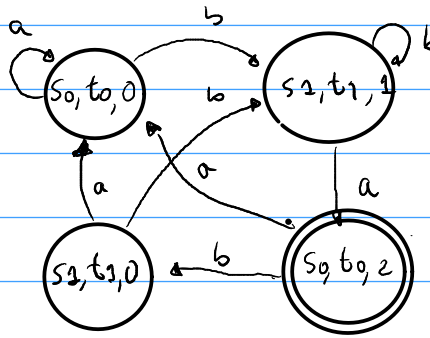
Ejemplo



$L(A_1) = \{ \sigma \mid \exists i \geq 0 \sigma(i) = b \}$

$L(A_2) = \{ \sigma \mid \exists j \geq 0 \sigma(j) = a \}$

$L(A_1 \cap A_2) = \{ \sigma \mid \exists i \geq 0 \sigma(i) = a \vee b \}$



Ejercicio 12. Demuestre que $\overline{L(A\phi)} = L(A\neg\phi)$.

$$\overline{L(A\phi)} = L(A\neg\phi) \quad \overline{L(A\phi)} = L(A^c\phi) = L(\neg\phi) = L(A\neg\phi)$$

$$L(\overline{A\phi}) \subseteq L(A\neg\phi)$$

$$L(A\neg\phi) \subseteq L(\overline{A\phi})$$

$$\forall \sigma \quad \sigma \in L(\overline{A\phi}) \Rightarrow \sigma \in L(A\neg\phi)$$

$$\forall \sigma \quad \sigma \in L(A\neg\phi) \Rightarrow \sigma \in \overline{L(A\phi)}$$

$$\neg \exists \sigma \quad \sigma \in L(\overline{A\phi}) \wedge \sigma \notin L(A\neg\phi)$$

$$\neg \exists \sigma \quad \sigma \in L(A\neg\phi) \wedge \sigma \notin \overline{L(A\phi)}$$

$$\neg \exists \sigma \quad \sigma \in L(\overline{A\phi}) \cap L(A\neg\phi)$$

$$\neg \exists \sigma \quad \sigma \in L(A\neg\phi) \cap L(A\phi)$$

$$L(\overline{A\phi}) = L(A\neg\phi)$$

$$\neg \exists \sigma \quad \sigma \in L(A\neg\phi \cap A\phi)$$

$$= L(A^c\phi)$$

$$\neg \exists \sigma \quad \sigma \in \emptyset = \text{True}$$

$$= L(A\phi)$$

Ejercicio 13. Recordemos que una fórmula es satisfactible si es verdadera en algún modelo. Dé un algoritmo para chequear que una fórmula LTL ϕ es satisfactible. (Pista: Piense en el algoritmo básico de model checking.)

Armar $A\phi$



Ver si hay un ciclo que alcance al estado de aceptación



si existe algún ciclo entonces ϕ es satisfactible

Ejercicio 14. Se dice que un modelo M refina a un modelo M' si todas las propiedades que satisface M también las satisface M' . Dé un algoritmo para chequear que un modelo M refina a un modelo M' . Justifique su respuesta. (En este ejercicio nos referimos a cualquier propiedad más allá de las expresables en LTL, es decir, a cualquier conjunto de ω -palabras.)

Queremos ver si las propiedades que satisface M las satisface M' , es decir si $L(M) \subseteq L(M')$

Además $L(M) \subseteq L(M')$ si? $L(M) \cap L(M') = L(M)$, por ende el algoritmo sería:

$$A_{\text{univ}} A_M \text{ y } A_{M'} \\ \downarrow \\ A_{\text{univ}} A_M \cap A_{M'} \\ \downarrow \\ \text{Verificar si } A_M \text{ es isomorfo a } A_M \cap A_{M'}$$

Ejercicio 15. Considere las proposiciones atómicas “reader $\bowtie i$ ” y “writer $\bowtie i$ ”, donde $i \geq 0$ y $\bowtie$ es $<$, $\leq$, $=$, $\geq$, o $>$, y que tienen el significado esperado. Por ejemplo la proposición reader $> i$ indica que hay más de i lectores en el sistema.

1. Use LTL para representar la propiedad de safety de lectores-escritores del Ejercicio 2 del Trabajo práctico 2
2. Dé una fórmula LTL que asegure fairness incondicional para los lectores y para los escritores.

1. $\Box \neg (\text{readers} > 0 \wedge \text{writers} > 0)$ no pueden haber lectores y escritores al mismo tiempo
 $\Box \neg (\text{writers} > 1)$ no puede haber más de un escritor a la vez
 $\Box \neg (\text{readers} > k)$ no puede haber más de k lectores a la vez
 2. $\Box \Diamond (\text{readers} > 0) \wedge \Box \Diamond (\text{writers} > 0)$
 Se puede dar el caso en que hayan escritores y lectores al mismo tiempo

Ejercicio 16. Considere dos usuarios, *Itchy* y *Scratchy*, y una sola impresora que ocasionalmente necesitarán utilizar. Sin embargo sólo un usuario por vez tiene permitido imprimir sus trabajos. Considere las siguientes proposiciones atómicas

- *Itchy.request*: indica que la *Itchy* solicita el uso de la impresora.
- *Itchy.use*: indica que la *Itchy* tiene el recurso impresora y puede utilizarlo.
- *Itchy.release*: indica que la *Itchy* libera la impresora para que cualquier otro usuario la utilice.

Los mismos predicados se definen para el *Scratchy*. Especificar en LTL las siguientes propiedades:

1. *Exclusión mutua*: Sólo un usuario puede usar la impresora en un momento dado.
2. *Tiempo de uso finito*: Un usuario no puede estar habilitado para usar la impresora durante un tiempo infinito.
3. *Ausencia de inanición individual*: Todo usuario que pretende imprimir algo, puede hacerlo, ocasionalmente, luego de esperar un tiempo.
4. *Acceso alternante*: Los usuarios deben alternarse estrictamente para hacer uso de la impresora.

1. $\Box \neg (\text{Itchy.use} \wedge \text{Scratchy.use})$

2. $\Box((\text{Itchy.use} \rightarrow \Diamond \text{Itchy.release}) \wedge (\text{Scratchy.use} \rightarrow \Diamond \text{Scratchy.release}))$
3. $\Box(\text{Itchy.request} \rightarrow \Diamond \text{Itchy.use}) \wedge \Box(\text{Scratchy.request} \rightarrow \Diamond \text{Scratchy.use})$
4. $\Box((\text{Itchy.use} \rightarrow \Box(\neg \text{Itchy.use} \vee \text{Scratchy.use})) \wedge (\text{Scratchy.use} \rightarrow \Box(\neg \text{Scratchy.use} \vee \text{Itchy.use})))$

Ejercicio 17. Considere un sistema de ascensores que sirve a N pisos ($N > 0$) numerados de 0 a $N - 1$. Los ascensores se acceden desde todos los piso, y en cada piso, junto a la puerta hay un botón para llamarlo y una luz que indica si el ascensor ha sido llamado o no. Dentro del ascensor hay N botones que permiten solicitar el traslado a cada uno de los pisos, y N luces indicadoras que informan los pisos solicitados (desde dentro del ascensor) pero aún no servidos. (Bah! un ascensor con memoria común y corriente).

Presente un conjunto mínimo de proposiciones atómicas necesario para modelar el problema de acuerdo a las propiedades siguientes. Dé, además, las fórmulas LTL que describen las siguientes propiedades:

1. Las puertas de los ascensores son *seguras* (*safe*) en el sentido de que nunca están abiertas si el ascensor no se encuentra en el piso correspondiente.
2. El requerimiento de ir a un determinado piso será atendido en algún momento.
3. El ascensor siempre retorna a la planta baja (piso 0).
4. Cuando hay un requerimiento del piso más alto (piso $N - 1$), el ascensor lo atiende inmediatamente sin detenerse en ningún lugar camino a ese piso.
5. El ascensor permanece inmóvil a menos que haya algún requerimiento.

piso[n]. solicitar $0 \leq n < N$
 piso[n]. llegar
 puertas[n]. abiertas

1. $\Box(\text{puertas}[n].\text{abiertas} \rightarrow \text{piso}[n].\text{llegar})$
2. $\Box(\text{piso}[n].\text{solicitar} \rightarrow \Diamond \text{piso}[n].\text{llegar})$
3. $\Box \Diamond \text{piso}[0].\text{llegar}$
4. $\Box(\text{piso}[N-1].\text{solicitar} \rightarrow \text{piso}[N-1].\text{llegar})$
5. $\Box(\neg \text{piso}[n].\text{solicitar} \rightarrow (\text{piso}[n].\text{llegar} \rightarrow \Box \text{piso}[n].\text{llegar}))$